NC STATE UNIVERSITY

ECE 592 : RESONANT POWER CONVERTERS

Lab - 1

Sanjana Ashok Bhalekar
Abhishek Ajit Deshpande
Raghav Chandrasekaran
Sydney Cheng

Prof.: Dr. Zeljko Pantic
Instructor: Zhansen Akhmetov

Table of Contents:

Content	Page. No
Introduction	3
Task 1	4
Task 2	7
Task 3	9
Task 4.1	11
Task 4.2	12
Conclusion	15
Appendix A	16

Introduction:

Resonant converters are critical components in modern power electronics, demanding robust sensing and protection subsystems for reliable operation. This report presents the results of a laboratory experiment designed to evaluate these subsystems using the NCP51810 HB GaN Driver Evaluation Board, driven by a Texas Instruments microcontroller. The experiment aimed to characterize the converter's behavior under varying operating conditions, specifically focusing on the impact of frequency and dead time adjustments, controlled through the Code Composer Studio (CCS) software. Additionally, the functionality of the overvoltage protection and output voltage fault sensing mechanisms was rigorously tested by simulating fault scenarios.

Task 1: Frequency Variation

Aim:

This task aims to configure and verify the operation of a microcontroller-controlled power circuit. This includes setting up the power supplies, programming the microcontroller, measuring the output frequency using an oscilloscope, and adjusting the frequency variable in the Code Composer Studio (CCS) debug window. The goal is to monitor and record the oscilloscope's switching frequency and document the results.

Test Setup:

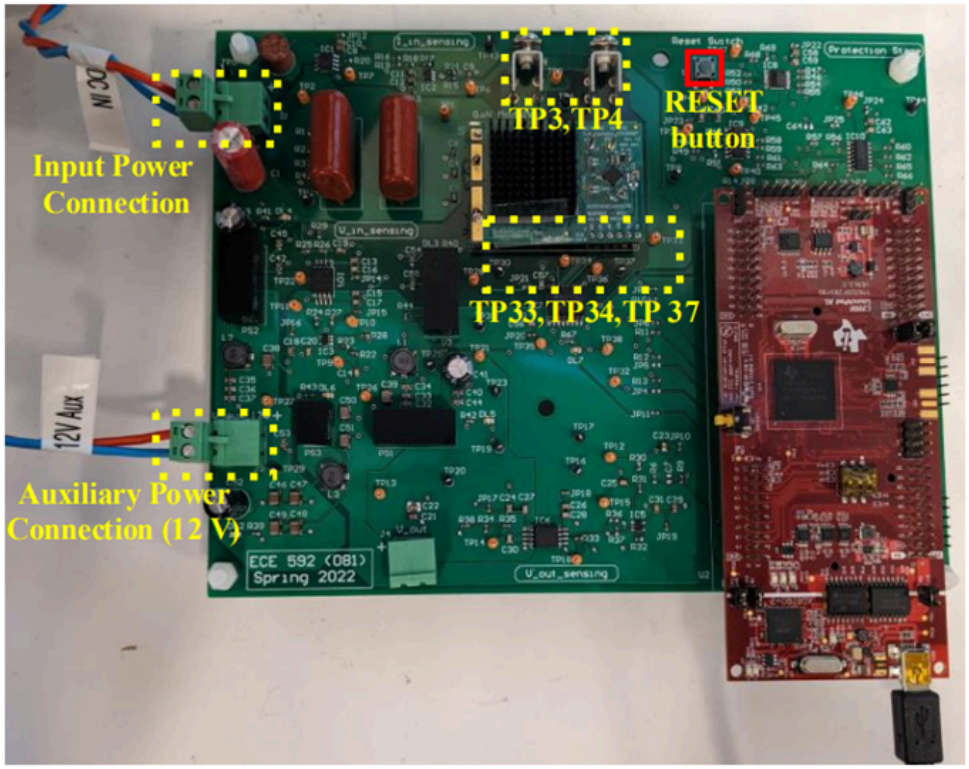


Fig. 1 Task 1 Setup connections

Procedure Conducted:

1. Initial Connections:
    - Connect both the input and auxiliary power sources (ensure both are in the OFF mode) and link the microcontroller to the computer via USB.
    - Connect one isolated voltage probe to the low-side PWM input pins (TP34 (+), TP37 (-)) and another probe to the output of the half-bridge (TP3 (+), TP4 (-)).
2. CCS Setup:
    - Open the Code Composer Studio (CCS) and the project file.
    - Add the frequency variable ‘Fref_kHz_input’ to the expressions window and confirm it reads 105 kHz.
3. CCS Debug Window:

- Activate continuous refresh in the CCS Debug window to monitor the variable's real-time value.
- Ensure the Debug window reflects the correct information.

4. Compile and Program the Microcontroller:
   - Compile the CCS code and reprogram the microcontroller.
5. Power Supply Configuration:
   - Set the auxiliary power supply voltage to 12V with a current limit of 0.2A, then power it on.
6. Reset Protection Button:
   - Pressed the 'reset' protection button on the PCB to clear the initial fault from the startup transient. The lit-up LED DL7 verified the reset.
7. Main Power Supply Setup:
   - Set the main power supply voltage to 0V and a current limit of 2A, then power it on.
   - Gradually increase the voltage until it reaches the nominal input of 48V.
   - Observe the output voltage on the oscilloscope. Use the oscilloscope's 'measure' functionality to measure the frequency and display it on screen.
   - Update the frequency variable in the Debug window to establish a 150 kHz switching frequency.
8. Oscilloscope Recording and Analysis:
   - Analyze the oscilloscope recordings to confirm the expected signal frequency.

Observations:

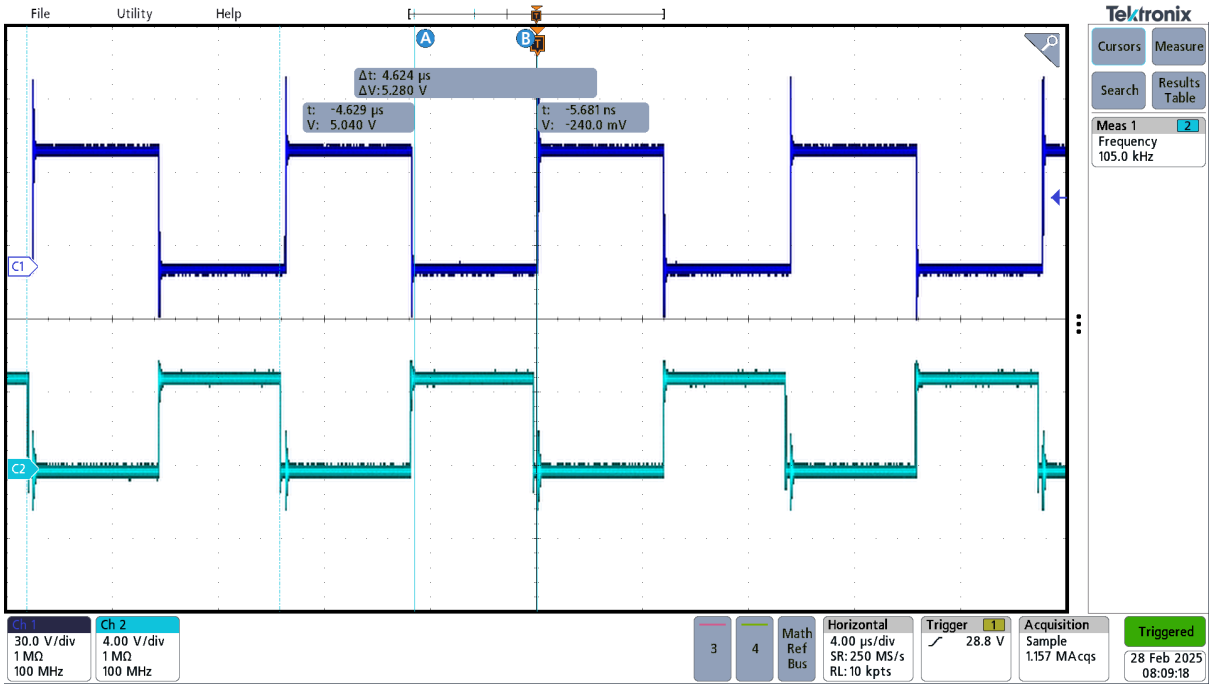


Figure 2. Voltage output waveform of TP3 and TP4 connections. The switching frequency is set at 105kHz. The dark blue waveform represents the PWM signal given to the system, and the light blue shows the waveform of the output voltage of the half-bridge circuit on the PCB.

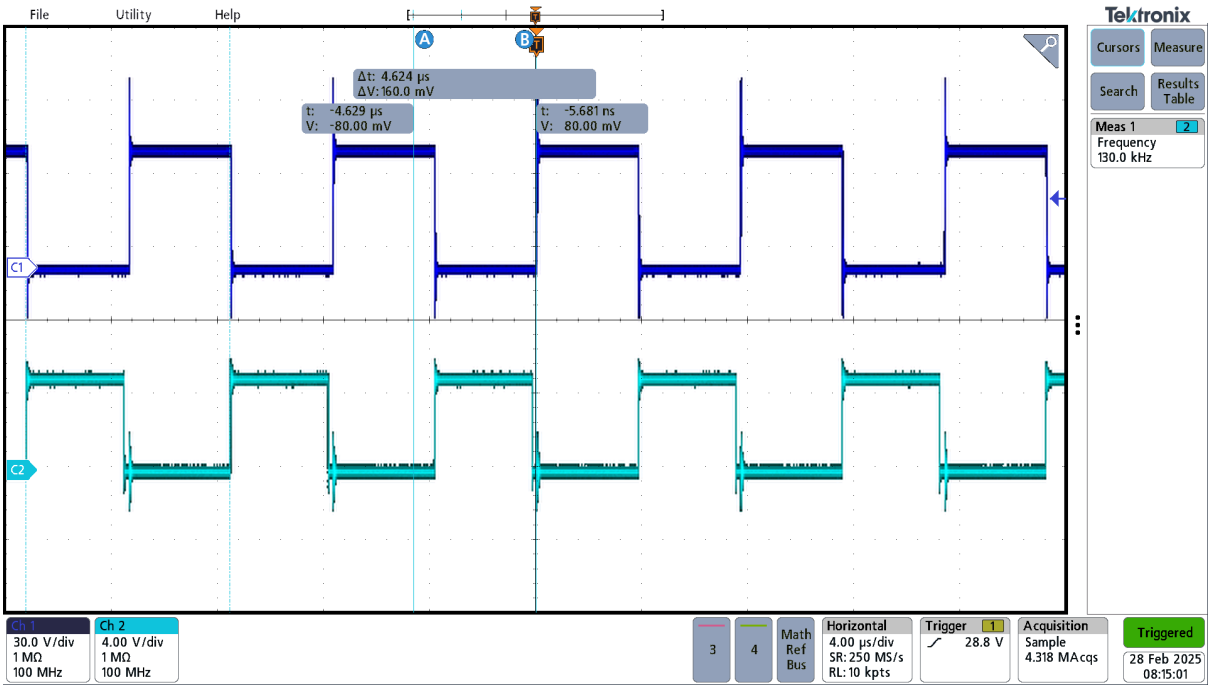


Figure 3. Voltage output waveform of TP3 and TP4 connections. The system was set to 150 kHz switching frequency. The dark blue waveform represents the PWM signal given to the system, and the light blue shows the waveform of the output voltage of the half-bridge circuit on the PCB.

Results:

Figures 2 and 3 illustrate the waveforms, which reveal that the half-bridge converter operates with an upper-frequency limit of 130 kHz. When the frequency was initially set to 105 kHz, the output waveforms corresponded to 105 kHz. However, when the input frequency was set to 150 kHz, the output waveforms stabilized at 130 kHz, indicating that the system's operational frequency is constrained to a maximum of 130 kHz.

Task 2: Dead Time Variation

Aim: To manually modify the dead-time in the gate signals using CCS in Debug mode and observe the corresponding changes in the oscilloscope measurements for an unloaded half-bridge circuit.

Test Setup:

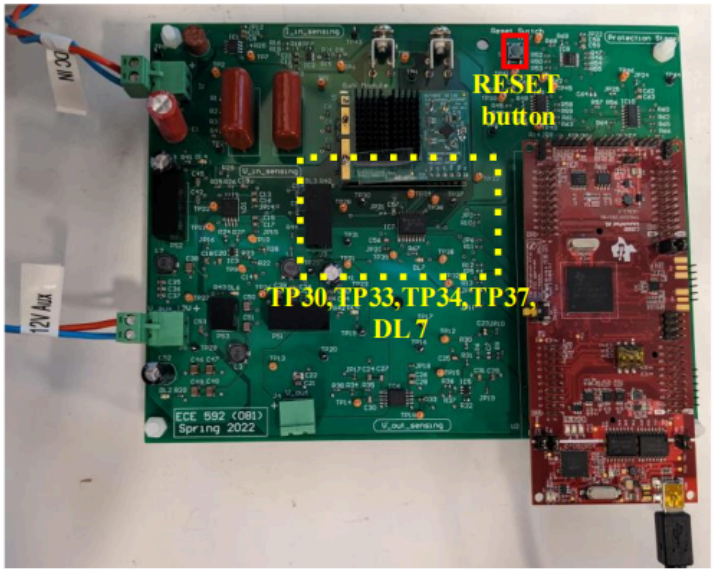


Fig. 11: Setup connections – Task 2.

- The connections on the board were made as shown in the figure above.
- An isolated voltage probe was attached to the high-side gate driver input (TP33+, TP37−) and another one to the low-side gate driver input (TP34+, TP30−).

Procedure Conducted:

- In the CCS IDE, while running the continuous debug mode, the variable ***DB_Val*** was initialised and its value was initially set to 32 to match 160 ns dead time between the falling and rising edges.
- Furthermore, the value of this variable was gradually changed in steps of 1 to see where it matches with 180 ns T_d.
- The value was found out to be 37 and the dead time was recorded to be 181 ns, close enough to what was expected.
- On the oscilloscope, the appropriate cursors were positioned along these edges to empirically monitor the dead time.

Observations:

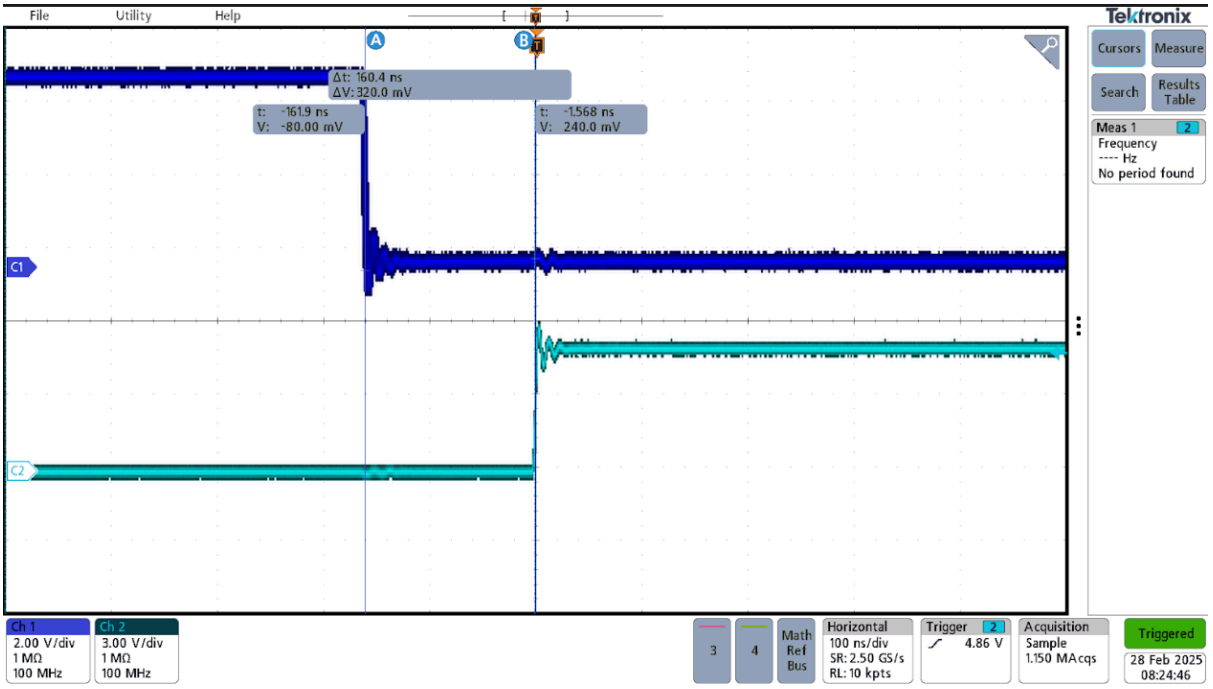


Fig. 5: 160ns Dead time

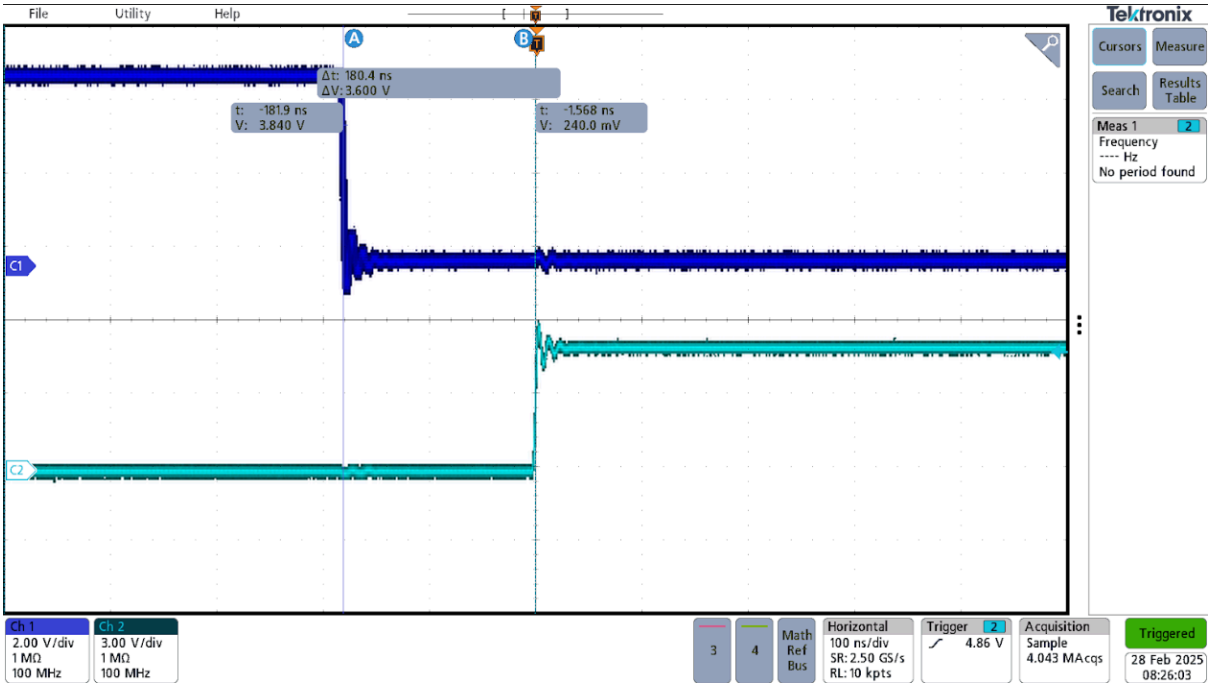


Fig. 6: 180ns Dead time

- However, there are some transients along the rising and falling edges, due to which the oscilloscope cursor must be carefully placed at the appropriate location for measuring the dead time accurately.

Task 3: Protection Subsystem Validation

Aim: This task will illustrate the implementation and operation of a typical protection system using the overvoltage protection of the input DC signal as an example. The threshold is currently set to trigger fault at 75 V.

Test Setup: The connections were made according to the given task steps

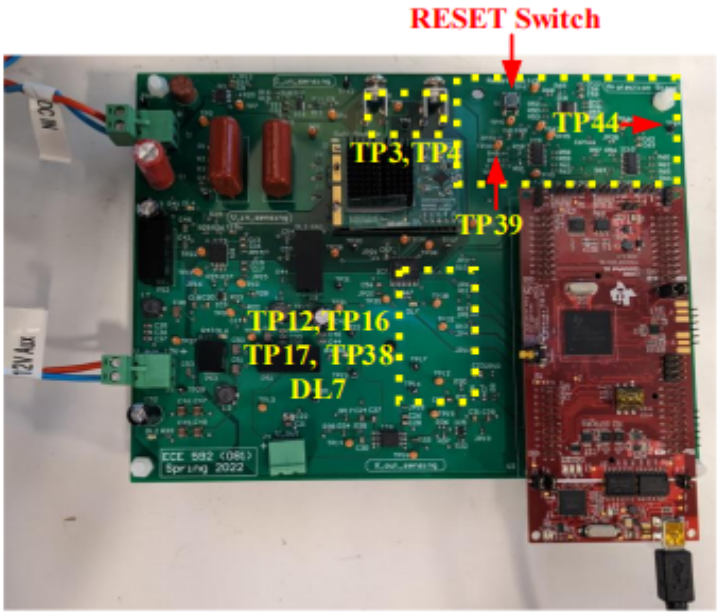


Fig. 13: Setup connections – Task 3.

Procedure Conducted: Three probes were used to make the connection

- The half-bridge output voltage (test pins TP3 (+) and TP4 (-)) (BLUE)
- The comparator output voltage (test pins TP42(+) and TP44(-)) (LIGHT BLUE)
- Fault Signal connected to the Disable_Gate_Driver pin of the gate driver (test pins TP38(+) and TP17(-)) (MAGENTA)
- The voltage was raised until 60V initially to verify that no fault occurs
- Then we set the fault signal trigger to **1V** initially,but we had to reduce it to a lower value of **0.92V** due to noise in the probes. This noise was minimized using ferrite bead
- We then continued Increasing the voltage and found out that the fault signal triggered at **76V**
- We then used the 'reset' to **enable the fault signal.** Ignoring this step does not produce the inverter out voltage waveform.

Results:

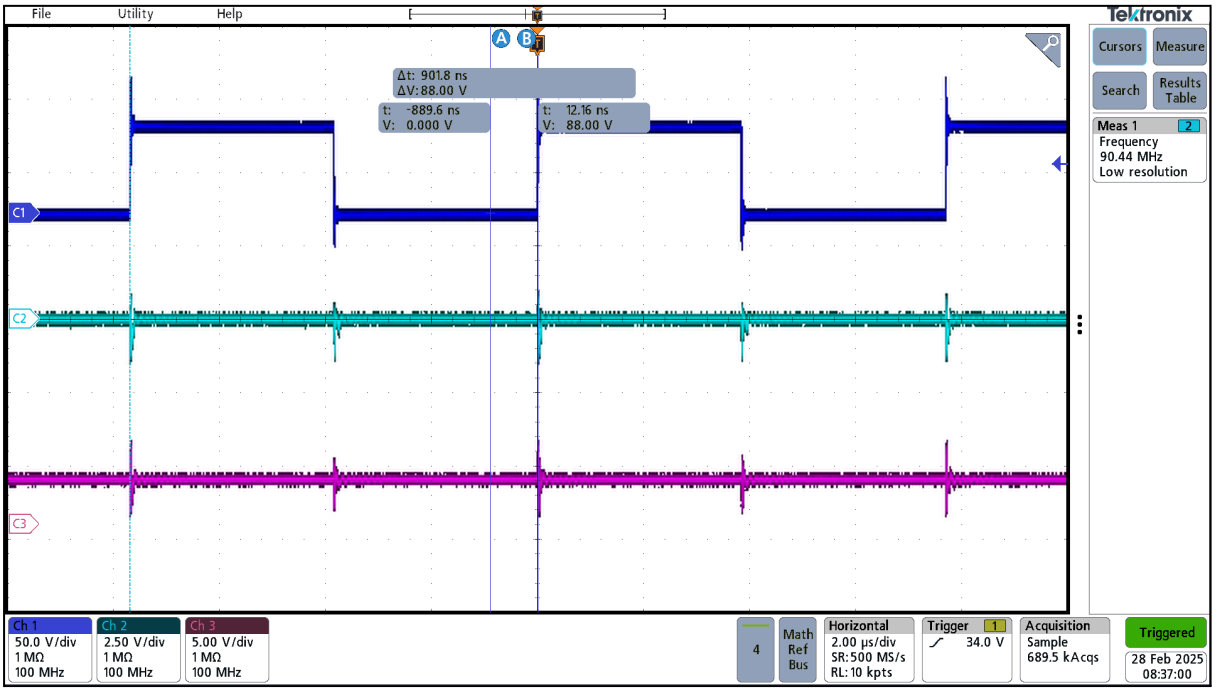


Fig 4.1: Waveform before fault i.e voltage raised to 60V

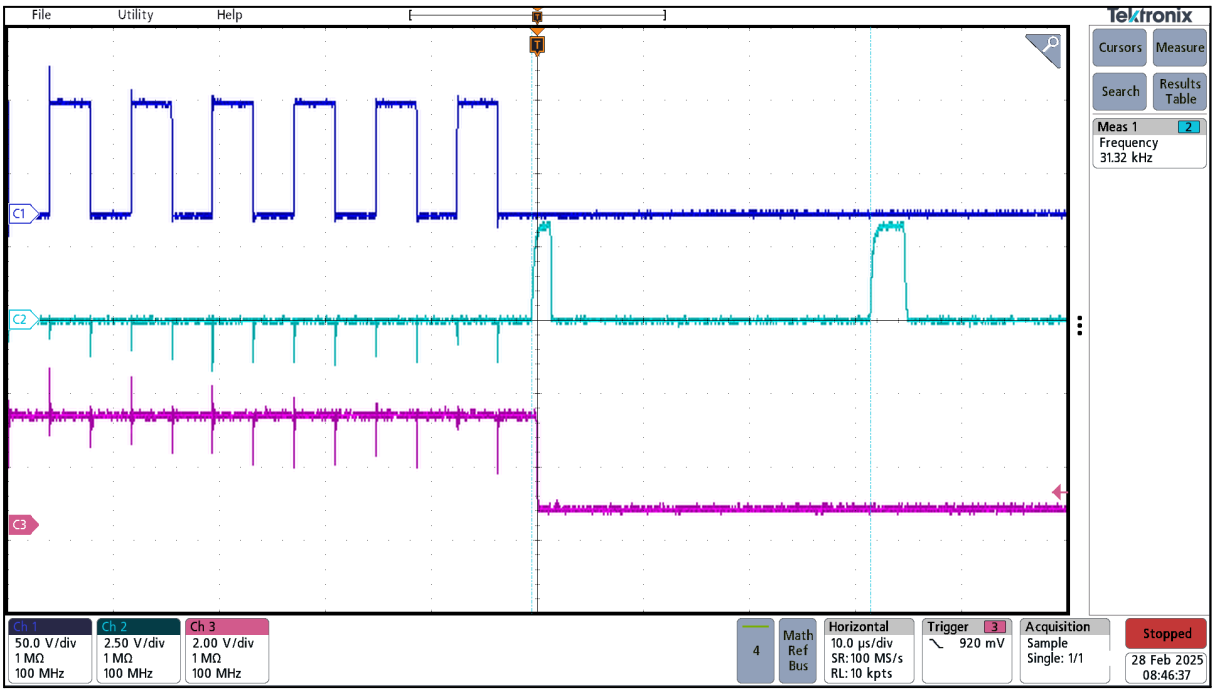


Fig 4.2:- During the Fault which occurred at 76 V

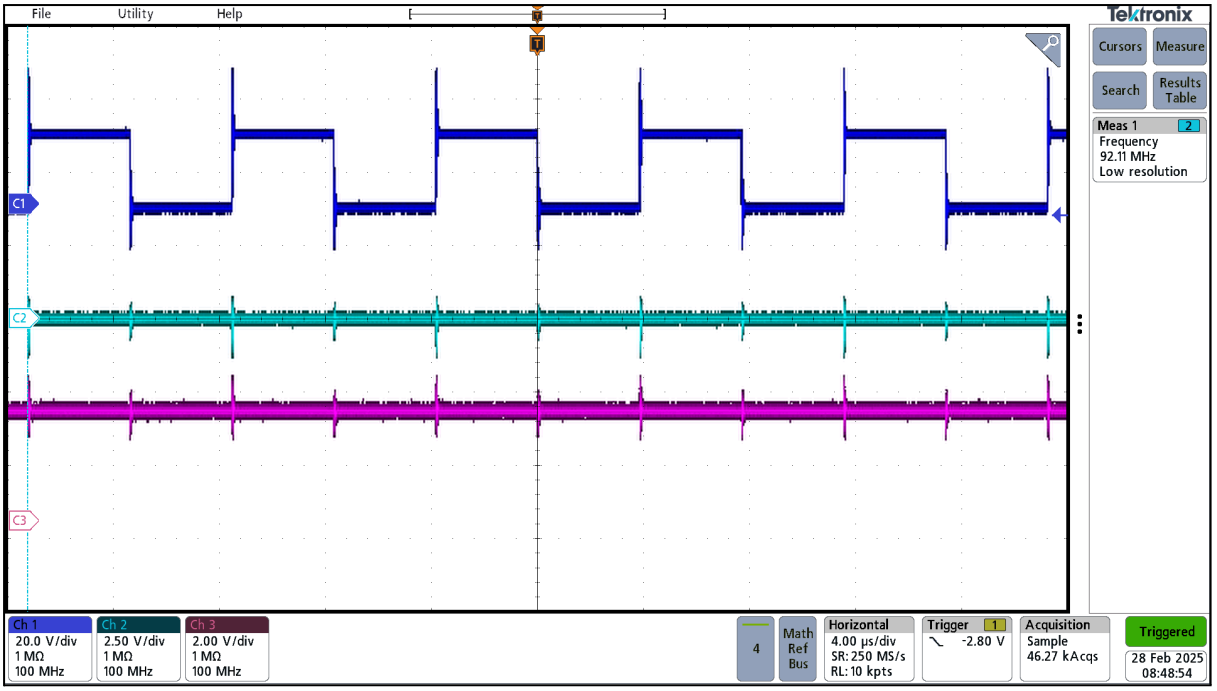


Fig 4.3: Waveform after using reset button to enable the fault signal and increasing the main input voltage

Task 4: Testing the Sensing Section

Task 4.1

Aim: Using the component values in Table I and the datasheets of the voltage sensor and the operational amplifier provided in the introductory section of this document, estimate the equivalent time constant τ_{vs}.

Calculations:

We know That,

$\tau vs = 1/2\pi f_H$,

$f_H \cong$ Band width(BW)

Rise time (tr)= 2.19* τ_{vs}.

$\tau_{vs} = \tau_s + \tau_{amp} + \tau_{f1} + \tau_{f2} + \tau_{f3}$

τ_{vs}= Equivalent Time constant

τ_s = Sensor time constant= $1/2\pi f_H$= **1.59 µ sec** f_H=higher cutoff frequency= 100KHz …(Using Datasheet)

τ_{amp} = Amplifier Time constant = $1/2\pi f_H$ = **0.199 µ sec** f_H=higher cutoff frequency=0.8MHz (Datasheet)

Table I: Component specification of the output voltage sensing unit

Component	Value	Component	Value
R_1	500 kΩ	R_6	2 kΩ
R_2	50 kΩ	C_1	2.2 nF
R_3	20 kΩ	C_2	100 nF
R_4	20 kΩ	C_3	10 nF
R_5	200 Ω	-	-

Now using Table 1 values time constants of three filters are as followed

τ_{f1}= (R1||R2)*C1= **100 µ sec**
τ_{f2}= R5*C2 = **20 µ sec**
τ_{f3}= R6*C3 = **20 µ sec**
Equivalent Time constant= 141.789 µ sec

Task 4.2: Measuring Slope and voltage offset

Aim: This task aims to measure the DC conversion function of the sensing section and calculate the parameter K_{VS} as a function of the measured voltage V_O
Test Setup: The connections were made according to the given task steps

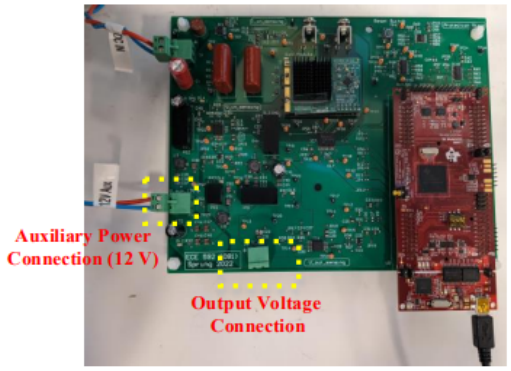


Fig. 14: Setup Connection – Task 4.

Procedure Conducted:

- The output Voltage was varied in steps of 1V to 20V and the Values were taken down from the CCS software expression.
- The reverse Order was followed to turn off the system

Observations: The values are put into a table and the slope was calculated using MATLAB

Voltage(V)	Digital Output Voltage	Reverse order values
0	**49**	**49**
1	**91**	**92**
2	**193**	**193**
3	**295**	**295**
4	**397**	**398**
5	**500**	**500**
6	**600**	**602**
7	**703**	**703**
8	**806**	**805**

9	**907**	**907**
10	**1010**	**1010**
11	**1112**	**1112**
12	**1214**	**1214**
13	**1316**	**1316**
14	**1418**	**1418**
15	**1520**	**1520**
16	**1622**	**1622**
17	**1723**	**1723**
18	**1825**	**1824**
19	**1927**	**1926**
20	**2028**	**2028**

Results:

The above values were put into MATLAB script and the slope and offset voltage were calculated.

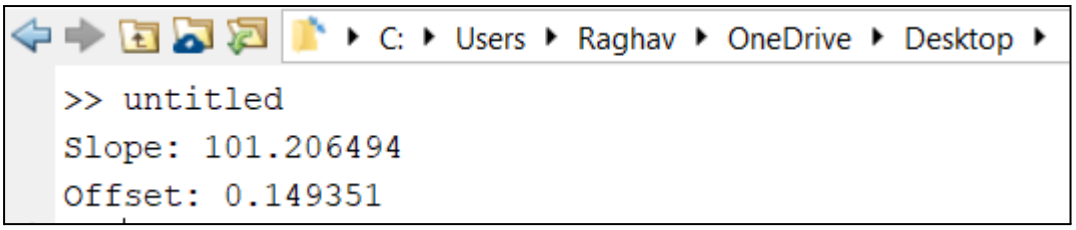


Fig 4.2.1 : Matlab Code Answers

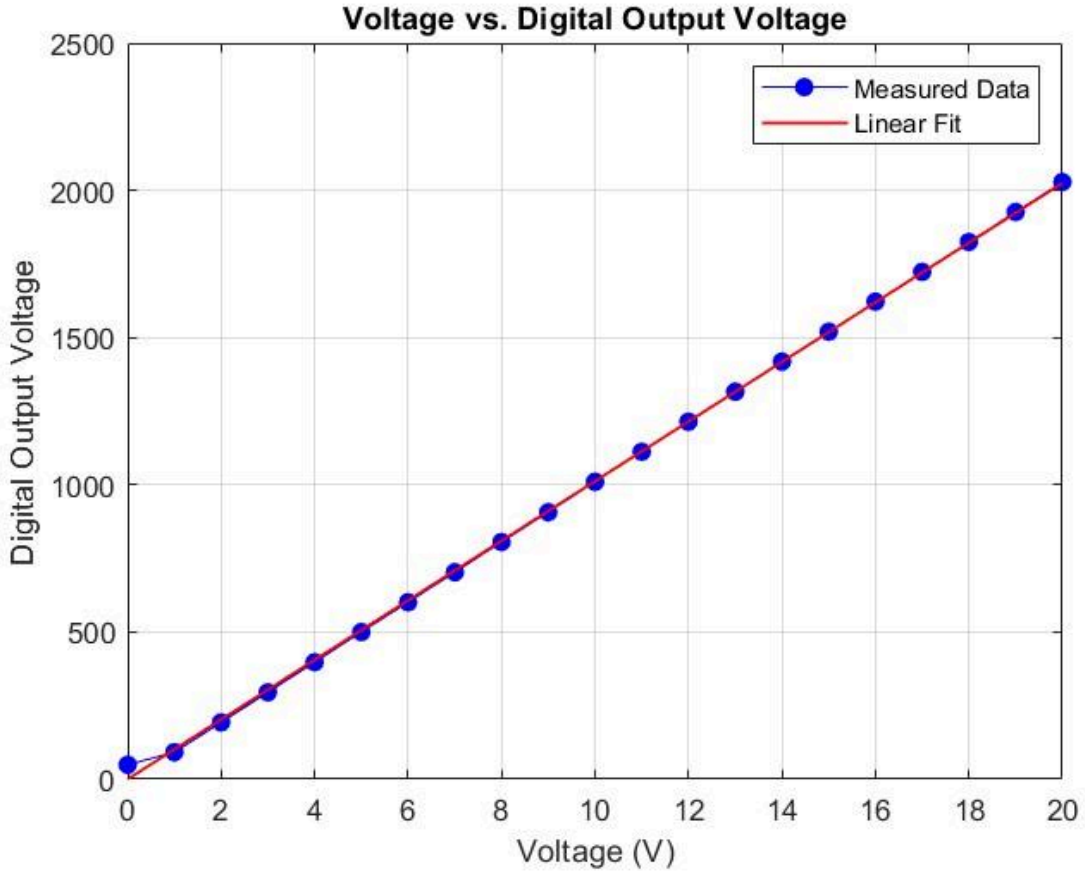


Fig: MATLAB plot for the above tabulated values

The average values were taken if the digital outputs had discrepancies

Conclusions:

- The first task involved the observation of frequency variation. It was observed that at 150KHz no output was observed. Through this we can infer that the frequency has an upper bound above which the output is not observed.
- The second task, the dead time, varied between 160ns - 180ns, by changing the number of bits in the rising edge and falling edge registers in the PWM module. The noise related effects were observed in the waveforms, which were corrected by setting the trigger value to 0.9.
- Upnext, the protection subsystem was tested under over voltage fault conditions by driving the main power above the set threshold voltage value of 75V.
- Finally, time constant and DC conversion function were measured using the given specifications and simulations based on empirical data.

APPENDIX A

Below is the Matlab code used to calculate the slope and the offset voltage

```
voltage = [0 1 2 3 4 5 6 7 8 9 10 ...
        11 12 13 14 15 16 17 18 19 20];
digitalOut = [49 91.5 193 295 397.5 500 601 703 805.5 907 1010 ...
          1112 1214 1316 1418 1520 1622 1723 1824.5 1926.5 2028];
p = polyfit(voltage, digitalOut, 1);
slope = p(1);
offset = p(2);
fprintf('Slope: %f\n', slope);
fprintf('Offset: %f\n', offset);
figure;
plot(voltage, digitalOut, 'bo-', 'MarkerFaceColor','b');
hold on;
fitLine = polyval(p, voltage);
plot(voltage, fitLine, 'r-', 'LineWidth',1);
xlabel('Voltage (V)');
ylabel('Digital Output Voltage');
title('Voltage vs. Digital Output Voltage');
legend('Measured Data','Linear Fit','Location','northeast');
grid on;
```